Day-2

Date __/__/____
Page _____

Modules

->modules are used in borrowing the codes of other peoples to use them in our python coding

There are two types of modules
1. Built in module
2. External module
- these are installed using pip

External modules
are the codes written by the other people who wants you to use codes in your coding we are not able to know what are the implementations of this codes but we are able to use it

Built in modules
are the modules which ships along with python programing language, there is No need for you to install them externally

Modules are way better than scratching in terms of errors because modules are written by experts and used by thousands of peoples
using modules consumes less time and effort

Date __/__/__
Page _____

Day - 3

Code :- Print :- Print is in python programming is function, a function to perform a task

In python programing the first program written is "Hellow world"

let us consider

Eg:-

Input		output
Print("go")	→	go
print("Hello world")	→	Hello world
Print(5)		5
print(Hello world)	→	Error!
print Hello world	→	Error!
print "Hellow world"	→	Error!
Print('Hellow world')		Hello world

Python,

Comments :-

A comment is a part of coding file that the programmer dosent want to exicute, rather the programmer can use them to explain a block of code or to avoide the execution of specific code while testing

Eg:- let us consider

Input		output
1 # Bilal	→	Hellow world
2 Print("Hello world")		

Date __/__/__
Page _____

is used to indicate that the scentence is a comment

Eg:

	Input	output
1	# this code is created by him	→ blablabla
2	print("blablabla")	

→ This is a single line comment

You can Comment in the same line where the code is

Eg:

	Input	Out Put
1	Print("Hello world")#winner	→ Hello world

1	Print("Hello world")	→ Hellow world
2	# Creator ----	

Multi line Comment

Eg:

	Input	out put
1	''' some thing ----	
2	print("----")	Hello world
3	is not necessary'''	→
4	print ("Hello world")	

''' ---- ''' is also indicator that the scentence is a comment

If you want to make a code as a comment simply press (Ctrl + /)

Eg: input
print ("I am king")
(press (Ctrl + /))

~~output~~ input
print ("I am king")

Some short cuts

1 "'I"'
press alt + ↓

1 "'
2 I"'

Escape sequence charectors
To insert charectors that cannot be directly used in a string we use an escape sequence charector

An escape sequence is a backslash (\) followed by the charectors

Eg ~~input~~ If the input is

Print("I am a good boy")

If you want to print like good boy

print ("I am a "good boy" ")

This is an error

Instead of using " directly use \"
like

printf ("I am a \" good boy\" ")

The output is

I am a "good boy"

~~another use of back slash (\)~~

Imp

	input		output
1	print ("lets play	→	Error
2	foot ball")		

1	print ("lets play foot ball")	⇒	lets play foot ball
2			

~~1 print("lets play football\n~~

~~1 printf" lets play\n~~

Some other use of Escape sequence

Input

1 printf("lets playfootball\n because It is a goodgame

(\n)

Output

lets play football

because It is a good game

Date __/__/__
Page _____

Seperator

Specify how to separate the objects if there is more than one. Default is " "

Input	output
print("bye", "no", "go")	bye no go
print("bye", "no", "go", sep="+")	bye+no+go

You can write anything instead of only (+)

Variables and data types

Variables is like a containers that holds data. Very similar to how our containers in kitchen

Variables are reserved for random value & large value. So they can written in a single variable like

	input	output
1	a = 1	3
2	b = 2	
3	Print (a+b)	

1	a = 12346a10	71086722
2	b = 58739812	
3	print (a+b)	

Date / /
Page

Data type

Data type specifies that the type of value a variable holds

In python, we can print the type of any operation using type function

If the input

```
a = 1
print(type(a))
```

output

<class 'int'>

('int' means integers)

there are some builtin data types

1) Numerical data :- 'int', 'float', complex

'int' :- 1, -8, 0 (Integers)

'float' :- 0.1, 0.001, -0.01 (decimals)

'complex' :- 6 + 7j

2) text data :- 'str'

str :- "Boss", "king" (String)

3) Boolean data :- 'bool'

Boolean data consist of values of true or false

4) Sequenced data :- 'list' + tuple

list :- Is an ordered collection of data with elements separated by a comma and enclosed within square brackets

list are mutable and can be modified after creation

list :-

a = [8, 2, 8.5, -7, ["bananna", "apple"]]

Tuple :- A tuple is an ordered collection of data with elements separated by a comma and enclosed within parentheses tuples are immutable and cannot be modified after creation

tuple1 = (("Parrot", "sparrow"))

5) Mapped data :dict

dict :- a dictionary is an unordered collection of data containing a key :value pairs. the key value pairs are enclosed with curly braces

dict = {"name" :"Sakshi", "age" :20, "can vote" :true}

Date __/__/__
Page _____

Operators

python has different types of operators for different operations. To create a calculator we require arithmetic operators

Operator	Operator name	Example
+	Addition	15+7
-	Subtraction	15-7
*	multiplication	15*7
**	Exponential	5**3
/	Division	5/3

Special operators

%	modulus	15%3

This shows the remainder when we divide 15 by 3

//	Floor division	15//07

out put will be without decimals
It means decimal will be gone

Type casting in python

The conversion of ~~dat~~ one data type into the other data type is known as typecasting (or) type conversion

python supports a wide variety of functions & methods like int(), float() str(), ord(), hex(), oct(), set(), tuple(), list(), dict() for type casting

ex:-

input

output

a = "1"

Date / /
Page

input
a = "1"
b = "2"
print(a+b)
a & b are in a string data type

output
12

If
a = "1"
b = "2"
print(int(a) + int(b))

3

There are 2 types of type casting
1) Explisit conversion
2) Implicit conversion

Explisit type casting
The conversion of one data type in to another data type, done via developer & programmer's intention & manually as per requirment it known as explisit type conversion

It can be achieved by using python's in built functions like int(), float(), hex()

Implicit type casting
data types in python do not have the same level ie. ordering of data types is not the same in python, some of the data types have higher order & some have lower order

Date / /
Page

while performing any operations with different data types in python, one of the variable's data types will be changed to the higher data type. According to the level, one data type is converted into other by python itself. this is called 'implicit type casting'

Taking user input in python

In python, we can take user input directly by using input() function. this input function gives a return value as string / character hence we have to pass that into a variable

~~variable = int(etc)~~

variable = input()

But input function returns the value as string. Hence we have to typecast them whenever required to another data type

Ex

variable = int(input())

variable = float(input())

We can also display a text using input function. This will make input() function take user input and display a message as well

Ex

a = input("enter the name:")

print a

If we put a name after the text

Date __/__/__
Page _____

Output will be

Enter name: Bilal
Bilal

What are strings?

In python, anything that you enclose b/w single or double quotation marks is considered a string. A string is essentially a sequence or array of textual data. string are used when working with unicode characters

Ex:

Input

```
a= "Harry"
print ("Hello," + a)
```

Output

Hello, Harry

It doesn't matter whether you enclose your string in single or double quotes the out put remains same

Ex

```
a = 'Harry'
print ('Hello,' a)
```

Hello, Harry

(or)

```
a = "Harry"
print ('Hello,' + a)
```

Hello, Harry

Sometimes the user might need to put quotation marks in b/w the string

Date __/__/__
Page _____

Ex

a = 'Iam a good boy'

If we write this directly it is an error

Ex :

Input	Output
a = 'I'm a good boy'	Error

Instead of using it directly we can use "escape sequence" (\) like

a = 'I\'m a good boy print(a)	I'm a good boy

(or) If you want to hilight good use like this also instead of escape sequence like

a = 'Iam a "good boy"' print(a)	Iam a "good boy"
(or) a = "Iam a 'good boy'" Print(a)	Iam a 'good boy'
(or) a = "I'm a good boy" print(a)	I'm a good boy
(or) a = "'Iam a good boy'" print(a)	'Iam a good boy'

Date __/__/__
Page _____

Multiline String :

If our string has multiple lines we can create them like this

```
1 a = """ do what ever
2 & go to hell "
3   I dont care """
4 print (a)
```

→

```
do what ever
& go to hell
I don't care
```

(or)

```
1 a = ''' do whatever
2 & go to hell
3   I dont care '''
4 print (a)
```

```
do whatever
& go to hell
I dont care
```

(or)

```
1 a = "do whatever \n & go to hell \n I dont
  care
2 Print a
```

```
do whatever
& go to hell
I dont care
```

\n (Escape sequence) to delay the execution of some part of the string

Accessing characters of string

In python, string is like an array of characters. We can access parts of string by using Index of the element in a words, Starting from '0' zero Square bracket can be used to access elements of the string

a = good

Index of good is

g	o	o	d
0	1	2	3

Date __/__/__
Page _____

Ex Input | output

```
a= good
print(a[0])
print(a[1])
etc
```

→ g
o

Looping through the string

we can loop through string using a for loop like this

This is for long strings

Ex

```
1 a = "go to hell"
2 for character in a:
3     Print (character)
```

output:
g
o
t
o
h
e
l
l

instead of this you can use any variable

This is a new variable that we don't want to write in input but we want vertical output

---

String slicing & operation on String

Length of the String

we can find the length of string using "len()" function

Ex

```
a= "king"
Print(len(a))
```

→ 4

Date __/__/__
Page

Operation on string / slicing string

as we know we can access any elements of a string by using [index]
Index of 1st element is zero
Index of 2nd element is 1
— — 3rd element is 2

Slicing of string

print(a[x:y]) means the elements from index x to y-1

let us consider that we want a certain part of a string (or) a certain elements then we can use [here : TO there]

Ex:-

```
a = "king"
print (a[0:4])
```

king
0 1 2 3

Here the here the output includes '0' but exclude '5'

[a:n-1]

(or)

```
a = "king"
print (a[0:3])
```

→ k i n
0 1 2

include ↓ exclude

(or)

```
a = "king"
print (a[0:2])
```

k i
0 1

If we didn't put '0' zero the pythyn interpreter act as if there is zero like

```
a = "king"
print (a[:4])
```

king
0 1 2 3

~~If we didn't put~~

Date __/__/__
Page _____

If we didn't put any Number on both side The Python interpreter will put zero at first and then the length of the string on the other side like

```
a = "king"
print(a[:])
```

king
0 1 2 3

Note :-

```
a = "king"
print(a[0:4])
```
→ king

is same as

```
a = "king"
print(a[0:len(a)])
```
→ king

is same as

```
a = "king"
print(a[:len(a)])
```
→ king

is same as

```
a = "king"
print(a[:])
```
→ king

negative-slicing

If we put -ve Numbers for slicing we get Ex :-

```
a = "king"
print(a[0:-3])
```
→ k

-3 means the NO subtracted from length of the string, which is 4-3 = 1

∴ [0:-3] = [0:1]

0 Included, 1 Excluded

Here when we put [0:-3] the python interpreter reads it like [0:len(a)-3]

∴ [0:-3] = [0:len(a)-3] = [0:4-3] = [0:1]

Let us solve some exercises

1) a="Apple"
print(a[-1:-3])
#what will the output

→ -1 means
len(a)-1=4

-3 means
len(a)-3=2

len(a)=5
∴ [-1:-3] = [4:2]
"That doesn't make any sens so there is no output"

2) a="Apple"
print(a[-3:-1]) → Pl

-3 means
len(a)-3=2

-1 mean
len(a)-1=4

len(a)=5
∴ [-3:-1] = [2:4]

~~output is~~

A P P l e
1 1 1 1 1
0 1 2 3 4

3) a="Harry"
print(a[-4:-8]) → ar
what will be the output

Date __/__/__
Page _____

len(a) = 5

H a r r y
0 1 2 3 4

5 - 4 = 1 , 5 - 2 = 3

[-4 : -2] = [1, 3]

output is ar

String methods

Since strings are immutable so Python provides a set of built-in methods that we can use to alter and modify the strings

1) upper():
This method converts all the elements to upper case (capital letters)

Ex

```
a = "king"
print(a.upper())
```

→ KING

2) lower():
This method converts all the elements to lower case (small letters)

Ex

```
a = "ABCDEF"
print(a.lower())
```

→ abcdef

Date / /
Page

3) strip(): This dose it's work in preffix and suffix specific

the strip() method removes any ~~characters~~ characters (such as space & symbol) before and after the string

Ex

```
a = "!!! Gretout !!!"                    → Gretout
print(a.strip("!"))
a = "  sstt  "                           → sstt
print(a.strip())
```

4) rstrip

The rstrip() removes ~~any~~ specific ~~character~~ charector in ther suffix.

like:

```
a = "!! Gretout !!"                      → !! Gretout
print(a.rstrip("!"))
a = "  Gretout  "                        →   Gretout
print(a.rstrip())
```

This because it only worked as suffix

5) replace():

~~The~~ this methode it used to replace specific charectors of a string

Ex

```
a = "sikver spoon"                       → silver Moon
print(a.replace("s","M"))
a = "I am a gental man"                  → I am agental ma
print(a.replace("a","o")
```

Date __/__/__
Page _____

6) split() :-
the split method splits the given string at the specified instance and returns the separated string as list items

Ex :-
a = "come lets play"
print (a.split(" "))
space
['come', 'lets', 'play']

a = "come lets play"
print (a.split("e"))
['com', 'l', 'ts play']

7) Capitalize :-
this method is used to turn the 1st letter to capital & turn the rest 1st letter to small letters

Ex
a = "king"
print (a.capitalize())
→ King

a = "kIng"
print (a.capitalize())
→ King

8) center(); it centeralizes the string with the given number of spaces.

Ex
str1 = "Yo bro"
print(str1.center(50))

output
25spaces Yo bro 25spaces

9) count()
returns the number of times the specific characters & string is repeated

Ex:
str1 = "thankyou bye bye"

print(str1.count("bye"))
output = 2

10) endswith()
returns boolean values based on wether the given string ends with the following character of string

Ex:
a = "Hi how are you?"
print(a.endswith("?"))

output = True

we can check for particular portion of string

Date __/__/__
Page _____

Ex:

print(a.endswith("?", 8, 16))

output = True

11) find()

it searches for the first occurrence of the given value and returns its index

if the given value is not present then returns -1

Ex: a = "He is a man, He is brave"

print(a.find(","))

output = 13

print(a.find("cow"))

output = -1

12) index()

it returns the index of the given character string

13) isalnum()

returns True if the string is only made of alpha numerical characters A-Z, a-z, 0-9

False if contains any other punctuations etc

14) isalpha()

returns True if the string is only alphabatic a-z, A-Z

false if contain any punctuations or symbols

15) islower()

return true if all the characters in lower case, false if not

Date __/__/__
Page _____

16) isprintable()
returns true if all the characters are printing
returns false if not
ex: str1 = ("Yo \n what's up")
print(str1.isprintable())
output = false (due to '\n' it is not being printed when we call str1)

17) isspace(): return true if string is only made of space false otherwise

18) istitle(): return true if every word of the string is capital else return false

19) isupper()
returns true if all characters are uppercase else false

20) startswith()
returns true if the string starts with given string else false

21) swapcase()
Changes character cases from lowercase to uppercase & from uppercase to lowercase and returns the string.

Date __/__/__
Page _____

22) a.title()

Converts the string to title case like every word of the string starts with capital letters

Conditional ~~statement~~ operators.

>, <, <=, >=, ==, !=

these operators are used in condition checking Conditional statements.

If elif else

lets understand this with an ~~exam~~ exampl

```
a = int(input("Enter your age"))
if (a >= 18):
    print("You can drive")
elif(a == 18):
    print("You can't drive")
else:
    print("You can drive.")
```

it simply checks the following condition if it's true then executes the code and skips other condition checking.

nested If - else statement.

using if else conditional checking in an other if else statement

Date __/__/____
Page _____

Time module (built in module)

→ Time module used to work with timestamps
- current time
- elapsed time

→ to introduce delays.
→ to measure execution time
→ To use time values.

① Importing time module.

~~import time~~
```
import time
```

common functions in time module.

a) 1) time.time() → Get the current time but gets the whole time in seconds.

to get it in the correct format of the time you should use

~~1) time.ctime()~~
2) time.ctime()
& use it like this
```
time.ctime(time.time())
```

3) time.localtime()
returns a structure time object that has information on time in detail.

Date __/__/__
Page _____

4) time.gmtime() - Get structure of time in UTC

5) time.strftime() → format time as a string

time.strftime("%Y - %m - %d %H - %M - %S", time.localtime)

(or)

time.strftime("%H:%M:%S")

(or)

time.strftime("%Y-%m-%D")

6) Introducing delays.

time.sleep(seconds)

Ex: time.sleep(3) → wait for 3 seconds.

7) Measuring execution time by using time.time()

```
start_time = time.time()
for i in range(100000)
    pass

end time = time.time()
print("execution time :", end_time - start_time)
```

2) using 'time.perf_counter()' → provides more accuracy than time.time()

```
import time
start = time.perf_counter()
for i in range (100000):
    pass
end = time.perf_counter()
print (start - end)
```

3) time.process_time() (It ignores sleep time & delay)
It shows CPU execution time)

```
start = time.process_time()
for i in range (100000)
    pass
end = time.process_time()
print (end - start)
```

4) Converting b/w time formats

1) Time.mktime() - convert structure time to time stamp

```
timestamp = time.mktime(local_time)
print ("timestamp:", time_stamp)
```

2) time.asctime() - convert struct_time to string.

```
print ("readable time:", tune.asctime(local
```

Date __/__/__
Page _____

Matchcase statements

Ex:-

```
x = int(input("enter the value of x: "))

match x:
    case 0:
        print("x is zero")

    case 4:
        print("a case is 4")

    case _:
        print(x)
```

wecase `case _:` is base case if the case dosent match any value then base case is printed.

Match case statement is similar to if else statements to checke the following cases.

here we can use if statements with default case

like

`case _ if x > 0:`

like this there can be multiple default cases.

Note:- (we don't need break statement to exit match statement unlike other coding languages.

Date __/__/____
Page _____

loops :-

there are always some codes we need to execute more than one time this we do by using loops.

for loop :-

it is used to iterate sequentially on iterateable objects

example : in strings it iterates over characters of string

in list it iterates over elements of the list.

```
name = "Naruto"
for i in name:
    print(i)
```

output

N
a
r
u
t
o

```
list_1 = ["Bold", "bat", "ball"]
for i in list:
    print(i)
```

output

Bold
bat
ball

we can use loops inside loop also known as nested loops.

Range

range() : function

we use this range function to provide range of elements in the loops.

Date __/__/__
Page _____

like :- for range(6) prints 0-5

```
for i in range(0, 6):
    print(i)
```

(output)
0
1
2
3
4
5

we use a 3rd parameter in range function which adds the 3rd parameter to the next iteration

```
for i in range(0, 10, 2):
    print(i, end = " ")
```

output :- 0 2 4 6 8

We usually use this to decrement the value

```
for i in range(5, 1, -1):
    print(i)
```

output :-
5
4
3
2

while loop :-

ex :-

i = 0

```
while (i <= 5):
    print(i)
    i += 1
```

output
1
2
3
4
5
6

Date __/__/__
Page _____

while loop is often used with conditional statements.
we can create both incremental & decremental loop
to create decremental loop

```
i = 6
while (count > 0):
    print (i)
    i -= 1
```

output: 6 5 4 3 2 1

we can use else statement after while loop also.

```
while ( ):
    # code
else:
    # code
```

here as soon as the while condition becomes false it control will enter else block

~~In Python there is no do while loop but we can emulate do while loop using~~

Date __/__/__
Page _____

Breake and continue statement

Some times ~~we need~~ while using loops we need the code to exit the loop at certain iteration & skip certain iteration and continue on next iteration, so to do this we use Breake and continue statement

Break:-

this statement is used to exit the loop at certain iteration

ex:-

I want it to exit at 5

```
for i in range(10):
    if (i==5):
        break
    print(i)
```

output
0
1
2
3
4

It didn't go after 4 because ~~I used~~ used Breake to exit loop

Continue:-

this statement is used to skip the iteration

Ex:-

I want to skip 5 and continue on next iteration

```
for i in range(10):
    if (i==5):
        continue
    print(i)
```

output:
0
1
2
3
4
~~skipped~~ 5
6
7

Date __/__/__
Page _____

In python there is no dowhile loop but we can emulate do while loop using an infinate while loop and a break statement

Ex:- i = 0

while true:
    print(i)
    i += 1
    if (10 % i != 0):
        break

output: 0

here 10 % 0 not 0 but even if it is false the loop ran for the first iteration.